



Cahier des charges — Application Taskly

1. Présentation générale

Nom du projet : Taskly

Type d'application : Application web de gestion de projets et de tâches (inspirée de Trello)

Équipe projet :

- **Younes Bahri**
- **Schekina Ahounou**

Contexte :

Dans le cadre d'un projet d'étude, l'équipe développe **Taskly**, une application web collaborative permettant la gestion de tâches sous forme de tableaux, listes et cartes, à la manière de Trello.

L'objectif est de concevoir une solution complète, ergonomique et fonctionnelle, intégrant un backend, un frontend et une base de données temps réel, avec déploiement sur le cloud et pipeline CI/CD.

2. Objectifs du projet

- Reproduire les fonctionnalités principales de Trello dans une version personnalisée nommée *Taskly*.
- Fournir une application web moderne, réactive et intuitive.
- Permettre la gestion de projets en équipe : création de tableaux, listes et cartes, suivi des tâches, commentaires et collaboration.
- Mettre en place une architecture technique professionnelle avec CI/CD, hébergement cloud et authentification sécurisée.

3. Périmètre fonctionnel

3.1. Authentification

- Connexion / inscription via Firebase Auth (email / mot de passe).
- Gestion de session et sécurisation des routes.
- Profil utilisateur basique (nom, photo, email).

3.2. Tableaux de projet

- Création, édition, suppression de tableaux.
- Attribution d'un titre et d'une couleur d'arrière-plan.
- Partage d'un tableau avec d'autres utilisateurs (collaboration).

3.3. Listes

- Chaque tableau contient plusieurs listes.
- Possibilité d'ajouter, renommer, réordonner et supprimer une liste.
- Drag & drop des listes pour modifier leur position.

3.4. Cartes (tâches)

- Chaque liste contient plusieurs cartes représentant des tâches.
- Création, modification, suppression et déplacement des cartes par glisser-déposer.
- Ajout de détails : description, labels, date limite, pièces jointes, commentaires.
- Marquage d'une carte comme terminée.

3.5. Collaboration

- Visualisation en temps réel (mise à jour instantanée grâce à Firestore).
- Attribution de membres à une carte.
- Système de commentaires et notifications dans le tableau.

3.6. Interface utilisateur

- Interface inspirée de Trello, minimaliste et responsive.
- Thème clair / sombre optionnel.
- Navigation fluide et transitions animées.

3.7. Administration (facultatif)

- Tableau de bord pour gérer les utilisateurs et projets.
- Statistiques d'utilisation.

4. Spécifications techniques

4.1. Architecture

- **Monorepo GitHub** contenant le frontend et le backend.
- Structure séparée :
 - `/apps/frontend` : React + TypeScript
 - `/apps/backend` : NestJS + TypeScript
 - `/packages` : types et modèles communs

4.2. Technologies utilisées

Composant	Technologie
Frontend	ReactJS + TypeScript
Backend	NestJS + TypeScript
Base de données	Firebase Firestore
Authentification	Firebase Auth
Hébergement	Google Cloud Platform
CI/CD	GitHub Actions
Gestion de versions	Git / GitHub
Gestion de projet	GitHub Projects

4.3. CI/CD

- Build et tests automatisés à chaque *push* ou *pull request*.
- Déploiement automatique du frontend et du backend sur GCP.
- Vérifications de lint, typage et couverture de tests avant merge.

4.4. Sécurité

- Authentification sécurisée via Firebase.
- Règles Firestore pour protéger les données utilisateurs.
- HTTPS obligatoire sur les environnements déployés.

5. Contraintes

- **Périmètre fonctionnel limité** à la reproduction des principales fonctionnalités de Trello (MVP).
- **Temps imparti** : durée du semestre universitaire.
- **Technologies imposées** : NestJS, ReactJS, Firebase, GCP.
- **Architecture** : monorepo GitHub.
- **Méthodologie** : gestion en cascade simple via GitHub Projects.

6. Organisation du projet

Rôle	Nom	Responsabilités principales
Développeur Fullstack	Younes Bahri	Développement backend (NestJS), développement frontend (React), intégration CI/CD, coordination
Développeur Fullstack	Schekina Ahounou	Développement frontend (React), développement backend (NestJS), intégration Firebase, collaboration CI/CD

Outils de collaboration

- **GitHub Projects** pour la planification (tâches, issues).
- **GitHub Actions** pour l'intégration continue.
- **Git / GitHub** pour la gestion du code source.

Méthodologie

- Cycle en cascade simplifié :
 1. Étude et conception
 2. Développement backend
 3. Développement frontend
 4. Intégration
 5. Tests et déploiement

7. Livrables

- **Application web fonctionnelle** déployée sur GCP.
- **Documentation technique** (installation, architecture, API, schéma Firestore).
- **Rapport de projet** décrivant les étapes, les choix techniques et les tests.
- **Dépôt GitHub** complet avec historique de développement, CI/CD, et tickets.